

Security Assessment

BuildPact

August 26, 2026

Overview

Project Summary

Project Name	BuildPact
Language	Rust (Anchor Framework)

Audit Summary

Delivery Date	August 26, 2026 UTC
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total
Critical	0
Major	0
Medium	1
Minor	2
Informational	1

Table of Contents

Summary

Overview

Project Summary

Audit Summary

Vulnerability Summary

Audit Scope (Scope includes all provided Rust files)

Findings

Finding-01 : Unprotected Vault Closure for Open Projects

Finding-02 : Redundant Parameter in create_project

Finding-03 : Manual Account Space Calculation

Finding-04 : Inaccurate Event Emission in distribute

Findings

4 Total Issues

Title	Category	Severity
Unprotected Vault Closure for Open Projects	Logical Issue	Medium
Redundant Parameter in create_project	Code Quality	Minor
Manual Account Space Calculation	Best Practices	Minor
Inaccurate Event Emission in distribute	Logging	Informational

Unprotected Vault Closure for Open Projects

Category	Logical Issue
Severity	Medium

Description

In the `close_project` instruction, a security check prevents a finalized project from being closed unless the vault balance is below a minimal dust threshold, ensuring members have been fully paid out. However, this threshold verification is restricted only to projects in the `Finalized` state.

If a user prematurely or accidentally transfers SOL to the vault PDA via a direct native transfer while the project is still in the `Open` state, the threshold check is entirely bypassed. This allows the project creator to immediately invoke the `close_project` instruction to dissolve the state and drain the entire vault balance into their own wallet, resulting in a potential loss of user funds.

Recommendation

It is recommended to enforce vault balance safety checks regardless of the project's state. The logic should be updated to strictly require that the vault contains no more than the rent-exempt minimum plus the designated dust tolerance before permitting closure, preventing the creator from sweeping inadvertently deposited funds from an open project.

Redundant Parameter in create_project

Category	Code Quality
Severity	Minor

Description

The `create_project` instruction accepts a `protocol_wallet` public key as an argument and subsequently validates it against a hardcoded `PROTOCOL_WALLET` constant.

Because the expected protocol wallet address is already statically defined within the program's source code, passing it via the instruction parameter is redundant. This implementation unnecessarily increases the transaction payload size and forces clients to supply data that the on-chain program already knows.

Recommendation

Remove the `protocol_wallet` parameter from the instruction signature. The program should directly assign the hardcoded constant to the project account during initialization, streamlining the instruction interface and preventing validation failures caused by client-side discrepancies.

Manual Account Space Calculation

Category	Best Practices
Severity	Minor

Description

The `Project` and `Member` data structures utilize manual byte calculations through a hardcoded `LEN` constant to dictate account space allocation during initialization.

While the current mathematical calculation is accurate, managing account space manually is error-prone. Modifying, adding, or removing struct fields in future iterations without adjusting the exact byte math could lead to insufficient account space constraints resulting in severe data corruption or failed deployments.

Recommendation

Adopt Anchor's provided macro for automatic space calculations to enhance maintainability and reduce the risk of human error. Applying this macro to the structs allows the program to dynamically compute space during compilation and seamlessly pass it to the account initialization constraint.

Inaccurate Event Emission in distribute

Category	Logging
Severity	☒ Informational

Description

Within the `distribute` instruction, the `FundsDistributed` event relies on the `after_fee` variable to log the total number of lamports allocated to the members.

During the distribution loop, individual member shares are computed using integer division, which inherently rounds down and leaves minor amounts of lamport dust in the vault. As a result, the aggregated total of lamports strictly transferred to the users is slightly less than the logged `after_fee` amount, causing the event to emit an imprecise representation of the executed transfers.

Recommendation

Consider accumulating the exact calculated transfer amounts within the distribution loop and logging this verified total in the event. Alternatively, document within the protocol's integration guides that the emitted distribution metric is a pre-rounded calculation and will include fractional dust retained by the vault.

Summary

This report outlines the security assessment of the BuildPact Solana Program, developed in Rust utilizing the Anchor framework. The assessment involved a rigorous evaluation of the provided source code employing Static Analysis and Manual Review techniques.

The audit prioritized the identification of potential vulnerabilities, architectural flaws, and adherence to established Solana and Anchor development best practices. Key focal points included access control boundaries, PDA validation, fund transfer mechanics, and mathematical precision during share distributions.

The examination surfaced four distinct issues across varying severities. The most critical discovery is a Medium severity logical flaw within the project closure mechanism, which inadvertently permits project creators to drain user funds from prematurely funded open projects due to bypassed state-checks. Additionally, Minor severity issues were pinpointed relating to manual account space calculations and redundant instruction parameters that could be refactored for better gas efficiency. An Informational finding regarding event logging precision was also documented.

The codebase demonstrates strong logical structuring and effectively implements checked arithmetic to prevent standard overflow vectors. Resolving the identified findings, particularly standardizing the vault closure requirements, will significantly fortify the program's security posture and ensure reliable user experiences upon deployment.

...

Disclaimer

For the purposes of this disclaimer:

- "*Noah AI*" refers to the product currently operated by its owners and developers, including but not limited to its creators, contributors, affiliates, officers, employees, contractors, and agents.
 - "*Plena*" refers to Plena Finance Digital LLC, including but not limited to its affiliates, officers, employees, contractors, and agents.
 - "*User*" refers to the recipient of this report, whether an individual or an entity, and any of its affiliates, officers, employees, contractors, or agents who access, review, or act upon the contents of this report.
-

AI-Generated Content

This report is *entirely generated by artificial intelligence (AI)* without human authorship, review, or verification by Noah AI or Plena. AI-generated content is inherently probabilistic and may contain inaccuracies, false positives, or false negatives. *The User is solely and fully responsible* for reviewing, verifying, and validating all information before relying on it or taking any action.

No Responsibility or Liability

To the maximum extent permitted by law, *Noah AI and Plena, and their respective owners, operators, affiliates, officers, employees, contractors, and agents*, **accept no responsibility or liability** for:

- The accuracy, completeness, legality, reliability, or security of this report
 - Any hacks, exploits, unauthorized access, security breaches, or vulnerabilities
 - Any loss of funds, data, or digital assets
 - Any direct, indirect, incidental, consequential, punitive, or special damages of any kind, *whether in contract, tort, or otherwise*, even if advised of the possibility of such damages
-

No Endorsement or Advice

Nothing in this report constitutes or should be interpreted as an endorsement, recommendation, disapproval, investment advice, financial advice, legal advice, or guarantee of performance.

No Guarantee of Regulatory Compliance

Nothing in this report constitutes a guarantee by Noah AI or Plena of regulatory compliance, legal validity, or adherence to applicable laws in any jurisdiction.

No Contract Formation

Use of this report does not create any contractual obligations between the User and Noah AI, Plena, or their respective owners, beyond those explicitly stated in a written agreement between the parties.

Third-Party Dependencies

References to third-party technologies, services, or platforms are for informational purposes only and do not constitute endorsements or assurances by Noah AI or Plena of their reliability, security, or legality.

Risk Disclaimer

Blockchain technology, AI systems, and cryptographic assets are highly experimental and carry substantial risks. The User remains solely responsible for conducting independent due diligence, obtaining professional advice, and ensuring compliance with all applicable laws and regulations before acting on any information in this report.